

VQE approach to Prime Factorization

on IBM's Noisy Intermediate Scale Quantum Computer

Replication & Extension of Sobhani, Chai, Hartung, Jansen — Physical Review A 111, 042413 (2025)

Course project by Mrudula A Mahindrakar, EE25S011

253

Largest n
(real hardware)

1048561

On a classical
computer

CVaR

Cost function
optimised

2

Optimizers
(COBYLA & NFT)

Motivation & Background

The Factorization Problem

- Classically hard — basis of RSA cryptography
- Shor's algorithm gives exponential speedup
- Largest factored by Shor on real hardware: 21

NISQ Era Approach

- NISQ devices: noisy, limited qubits, no error correction
- VQAs excel here: small circuits + classical optimizer
- VQE finds ground state of a Hamiltonian

This Work

- Encode factorization as: $\min (n - pq)^2$
- CVaR-VQE on IBM hardware & simulator
- No arithmetic simplifications — fairest benchmark

Factorization Records

21 Shor on real hardware 2021

291,311 Adiabatic (NMR, 3 qubits) 2017

1,829 QITE on IBM (9 qubits) 2021

253 VQE — this paper (9 qubits) 2025

1,048,561 VQE — ideal simulation 2025

Method: CVaR-VQE Pipeline

1

Hamiltonian

Encode $(n-pq)^2$ into Pauli-Z operators.
Strip fixed LSBs $p_0=q_0=1$ to minimise qubit count.

2

Ansatz

Hardware-efficient linear-CNOT circuit.
L layers of RY gates + CNOT chain.

3

CVaR

Take lowest α -fraction of shot eigenvalues.
 $\alpha=0.01$ outperforms simple expectation.

4

COBYLA

Gradient-free simplex optimizes θ params until it converges or reaches max iters = $50 \cdot N \cdot L$

The problem with a standard VQE -

It calculates the average of all measurement values into the energy function $\langle H \rangle$.

Standard averages can lead to Barren Plateaus, where optimisers may fail.

Solution: CVaR only considers the relatively low-energy shots for the cost functions

$$\text{CVaR}(\alpha, S) = (1/\alpha S) \cdot \sum e_k$$

 e_k - ascending order of eigenvalues for S shots to obtain S measurement outcomes.
For $\alpha=1$, $E = \langle \varphi(\theta) | H | \varphi(\theta) \rangle$ is recovered

Cost Function Evaluation

Hamiltonian Eigenvalues · CVaR · Shot-Based Optimisation

CVaR with small α maximises the chance of observing the correct factors in at least one shot — the natural metric for the factoring problem.



Step 1 — Constructing the Hamiltonian

$$P = 1 + \sum_{j=1}^{N_p-1} 2^{j-1} (I - Z_{p,j}) \quad \text{and} \quad Q = 1 + \sum_{k=1}^{N_q-1} 2^{k-1} (I - Z_{q,k})$$
$$H = (nI - PQ)^2$$

Using this in our cost function evaluation, we have

$$H|p, q\rangle = \lambda_{p,q}|p, q\rangle = (nI - PQ)^2|p, q\rangle$$

Each measured bitstring $|p, q\rangle$ has eigenvalue $\lambda_{p,q}$.

Step 2 — CVaR Cost Function

The quantum circuit is measured S times, producing S eigenvalues $\{e_1, e_2, \dots, e_S\}$.

Sort the S eigenvalues in ascending order.

Take the mean of only the lowest $\alpha \cdot S$ values:

$$\text{CVaR}(\alpha, S) = (1/[\alpha S]) \cdot \sum e_k \quad [k=1..[\alpha S]]$$

Choosing α — The Tradeoff

Smaller $\alpha \rightarrow$ focuses on lowest-energy shots $\rightarrow$ better for finding the solution. But $\alpha \rightarrow 0$ makes the landscape discontinuous and the CVaR estimate noisy

Why CVaR Instead of $\langle H \rangle$?

Consider factoring $n = 9$:

$$|\psi_1\rangle = \sqrt{0.9}|00\rangle + \sqrt{0.1}|11\rangle$$

$$\langle H \rangle = 57.6 \quad | \quad \text{CVaR}(0.1) = 0.0 \quad \checkmark$$

10% chance of correct answer

$$|\psi_2\rangle = \sqrt{0.5}|01\rangle + \sqrt{0.5}|10\rangle$$

$$\langle H \rangle = 36.0 \quad | \quad \text{CVaR}(0.1) = 36.0 \quad \times$$

0% chance of correct answer

Classical VQE prefers $|\psi_2\rangle$ (lower average energy) but CVaR correctly identifies $|\psi_1\rangle$ as better — it has a 90% chance of yielding the solution in a single measurement.

Bitstring Encoding $|p, q\rangle$

State: $|p_{Np-1} \dots p_1 \ q_{Nq-1} \dots q_1\rangle$ with $p_0 = q_0 = 1$ implicit.

Example: $|010001\rangle \rightarrow p = 0101 \rightarrow p=5, q = 0011 \rightarrow q=3$

Ground state of H if $n = 15$, since $(15 - 5 \times 3)^2 = 0$

Linear-CNOT Ansatz Architecture

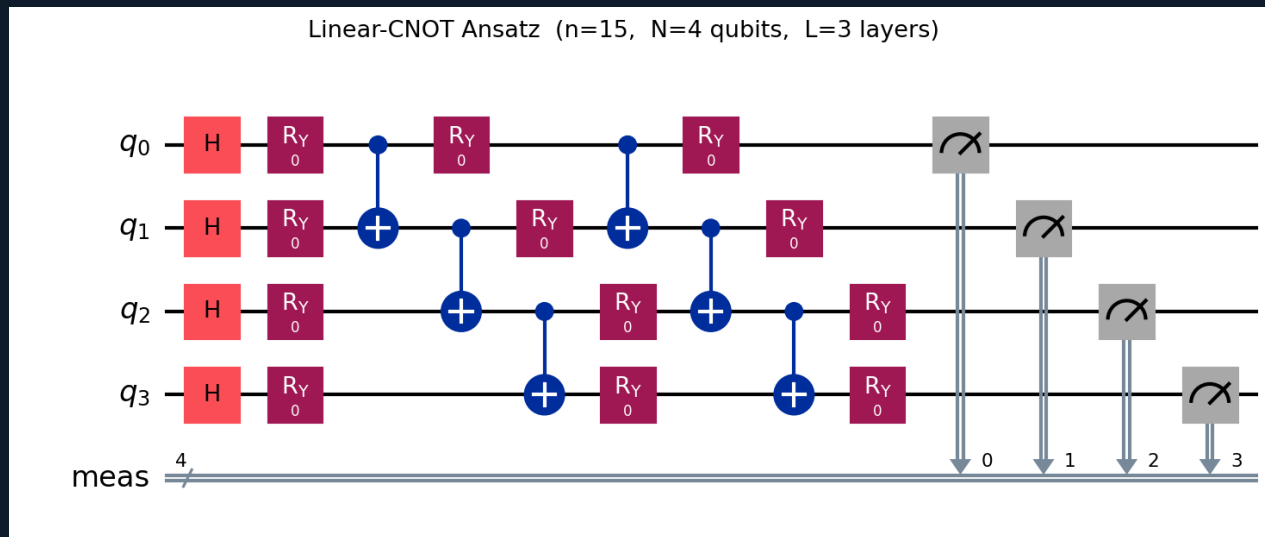


FIG. 2. Linear-CNOT Ansatz for $L = 3$.



Hadamard Init

Start in equal superposition $|+\rangle^N$



RY Rotations

$N \cdot L$ variational params $\theta \in (-\pi, \pi)$



CNOT Chain

$(N-1) \cdot (L-1)$ entangling gates

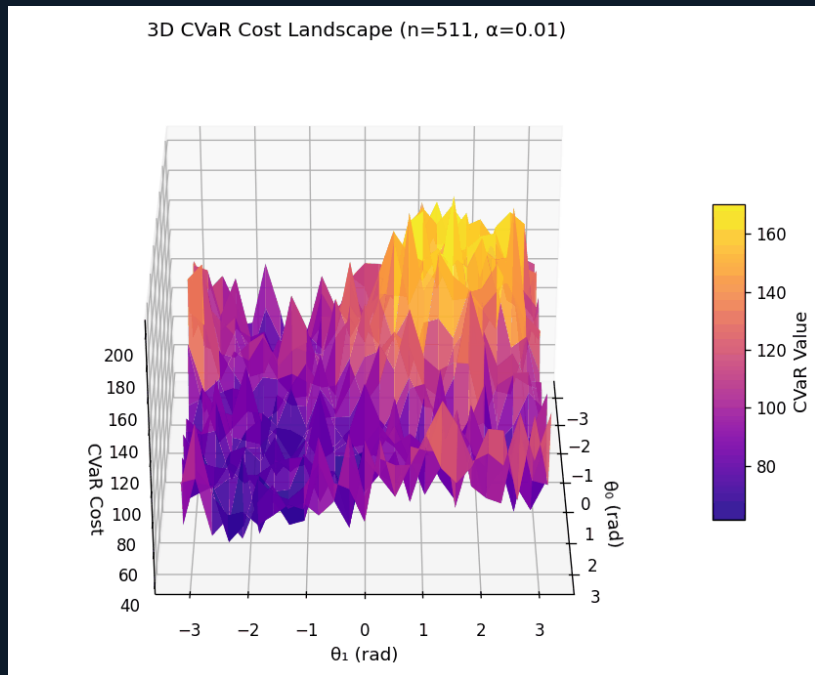


Measurement

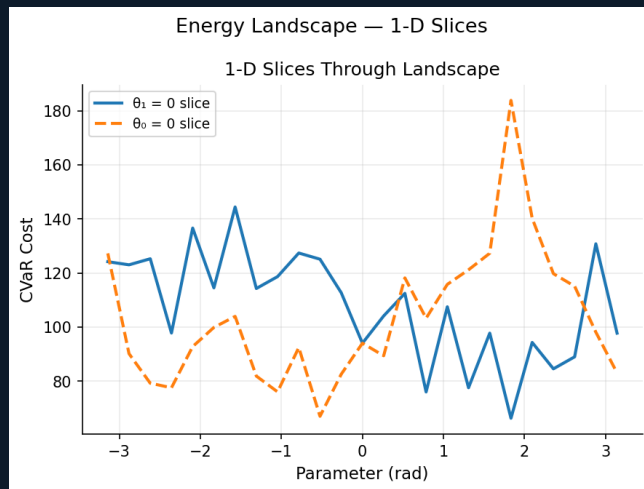
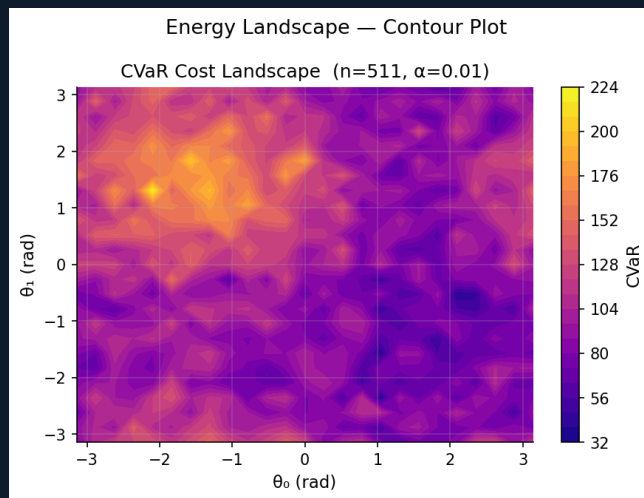
S shots $\rightarrow$ bitstring $\rightarrow (p, q)$ candidate

$$N = (N_p - 1) + (N_Q - 1) \text{ free qubits} \mid p \leq \sqrt{n} \rightarrow N_p = \lceil \log_2(\sqrt{n}) \rceil \mid p \leq q$$

CVaR Energy Landscape

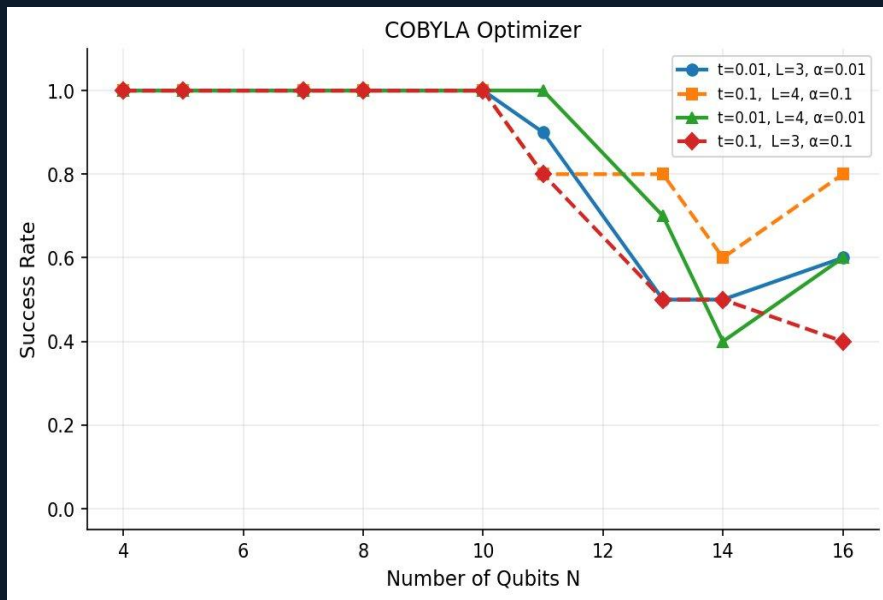


- Extremely rugged surface — hallmark of barren plateau
- Low-cost basins are tiny, isolated — no smooth funnel
- 1D slices oscillate 10–15× within a fraction of a radian



Success Rate vs Problem Size

COBYLA Optimizer



NFT Optimizer

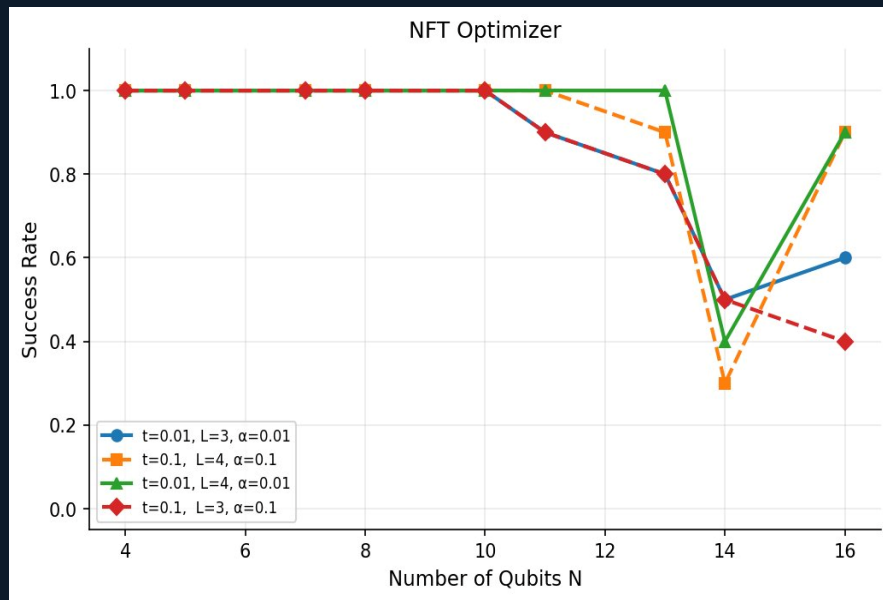


FIG. 3. Success rate for largest numbers n requiring N qubits

100% success for $N \leq 10$ qubits with both optimizers.

NFT clearly outperforms COBYLA at $N \geq 13$.

Anomaly: Both optimizers' performance dip at $N = 14$.

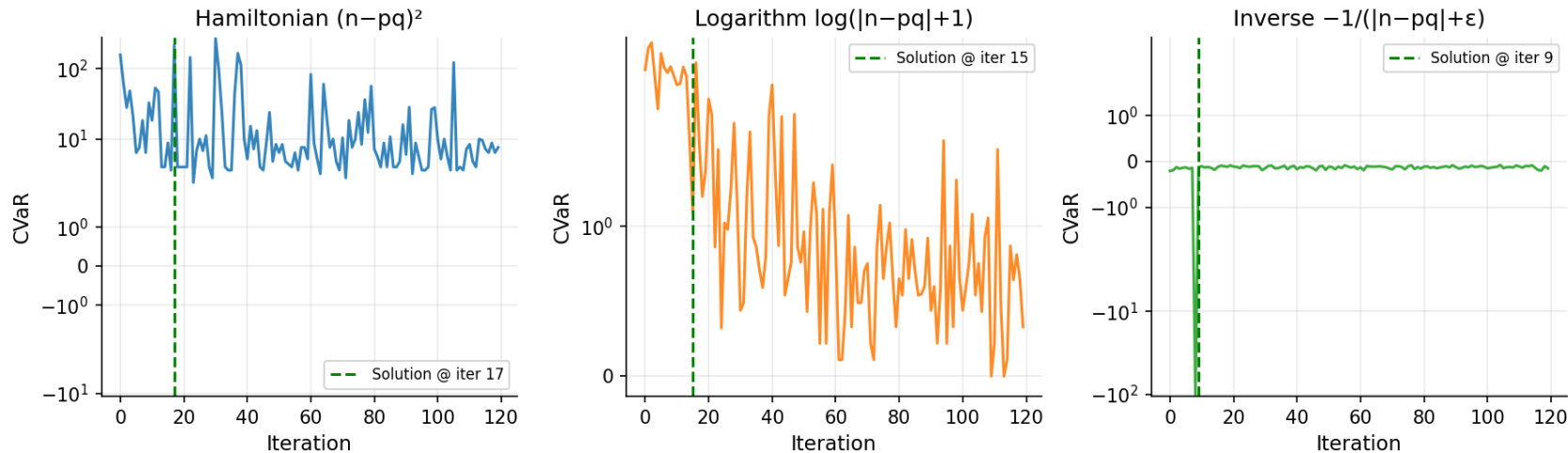
$\alpha = 0.01$ dominates $\alpha = 0.1$ throughout.

$L = 4$ has better success rate compared to $L = 3$ in both cases

Cost Function Convergence (n=511, 10k shots)

→ 1000 shots: solution found early but CVaR continues to bound - inverse cost locks in fastest.

Convergence Trajectory for n=511 ($\alpha=0.01$, shots=1000)



Hamiltonian $(n-pq)^2$

Converges at iteration 17, but CVaR plateaus around 10^1 not 0.

Barren Plateau.

Need to Increase L or shots to break through.

Logarithm $\log(|n-pq|+1)$

Finds solution at iter 15.

CVaR descends but remains noisy.

Best pairing with COBYLA with moderate shot counts.

Inverse $-1/(|n-pq|+\epsilon)$

Reaches solution earliest at iter 9.

CVaR is most stable post-convergence.
Fastest with NFT optimizer.

Iterations to Fidelity Threshold (n = 253)

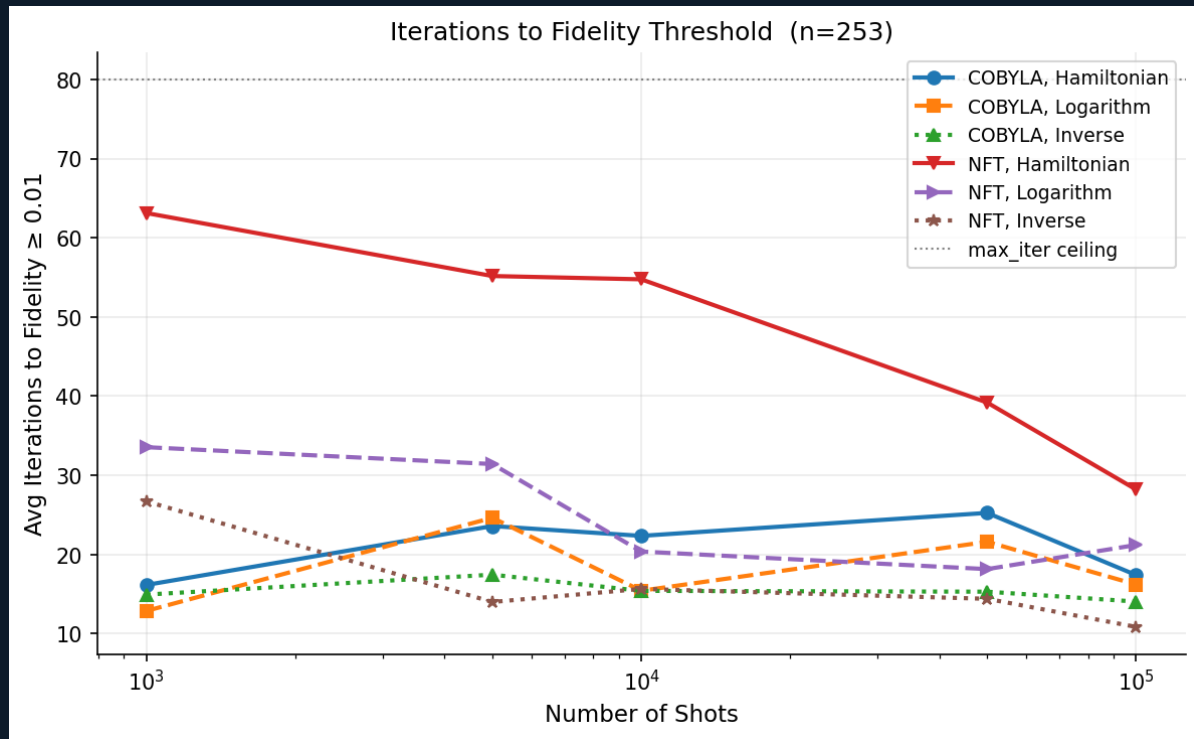


FIG. 10. Number of iterations required to reach the fidelity threshold for the successful runs, with various types of cost functions and using COBYLA or NFT optimizers.

Inverse + NFT wins long-term

NFT + Inverse (brown dashed) consistently reaches fidelity in ~ 10 – 15 iters across all shot counts, and improves as shots increase. Most reliable combination overall.

COBYLA + Inverse stable

COBYLA, Inverse (green triangles) shows remarkable stability, maintaining a low iteration count ~ 15 regardless of the number of shots.

NFT + Hamiltonian improves majorly, but is inefficient at lower shots

NFT, Hamiltonian (solid red) shows the most dramatic improvement, dropping from over 60 iterations at 10^3 shots to 30 iters at 10^5 shots. But at lower shots, it takes significantly higher iters to converge.

max_iter ceiling (dotted line = 80)

Hardware Results - IBM fez vs Simulator

$n = 253$ (11×23) · $N = 10$ qubits · *ibm_fez* · shots = 2000

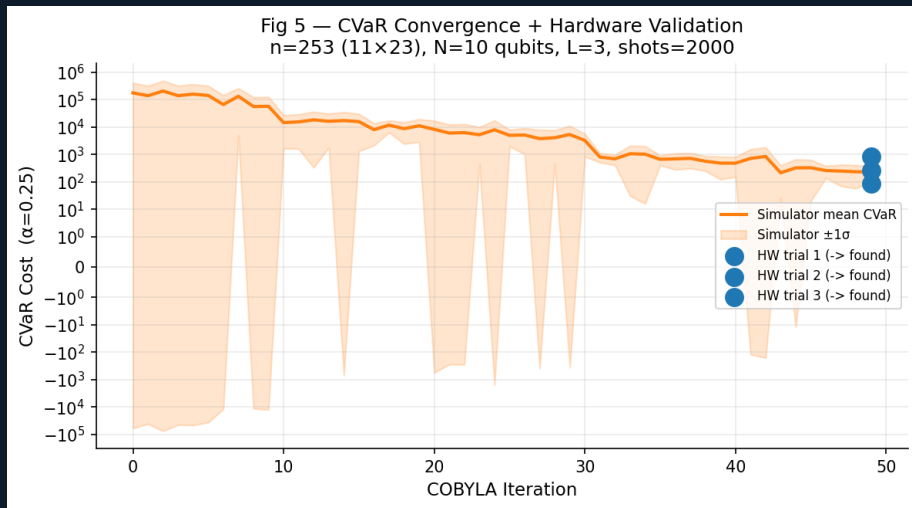
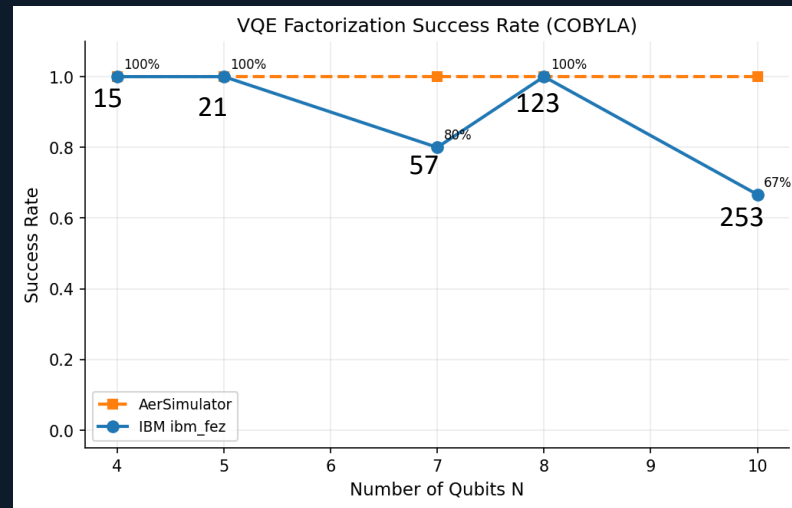


FIG. 5. Hardware run for $n = 253$, three layers, $\alpha = 0.25$, 2000 shots requiring ten qubits.



▼ $n=253$ drops to 67% on hardware.

Simulator envelope ($\pm 1\sigma$)

Orange band shows run-to-run variance across multiple simulator trials. Wide band at early iters reflects sensitivity to random initial parameters.

Hardware trials (blue dots)

All 3 IBM *ibm_fez* trials found the solution (11×23). Final CVaR values land within the simulator $\pm 1\sigma$ band — hardware noise is not catastrophic.

▲ Noise gap at $N=7$.
Gate error accumulates with circuit depth.

★ Hardware result confirms the paper's core claim: VQE factorization is viable on real NISQ devices up to $n=253$.

Key Findings & Next Steps

What We Found

- ✓ $\alpha = 0.01$ outperforms $\alpha = 0.1$ even with finite shots — contrary to naive intuition
- ✓ NFT beats COBYLA at $N \geq 13$ and with inverse cost function
- ✓ CVaR effectively overcomes Barren Plateaus by focusing on low energy shots
- ✓ 10k shots creates a phase transition: chaotic → convergent optimization
- ✓ Hamiltonian cost plateaus at ~ 15 (barren plateau, not shot noise)
- ✓ Logarithm + COBYLA: best for moderate shot budgets
- ✓ Inverse + NFT: fastest convergence at 10k+ shots

Future Work

1

Warm-Start Initialization for Inverse Cost Functions

The paper identifies - finding good initial parameters for the inverse-type cost as an open problem.

2

Barren Plateau Mitigation

Inverse cost function may suffer from barren plateaus (exponentially vanishing gradients).

3

Convergence Speed of Inverse Cost

Inverse cost needs more iterations than Hamiltonian cost. A cost switching strategy by training one model and feeding the parameters to another model can help.

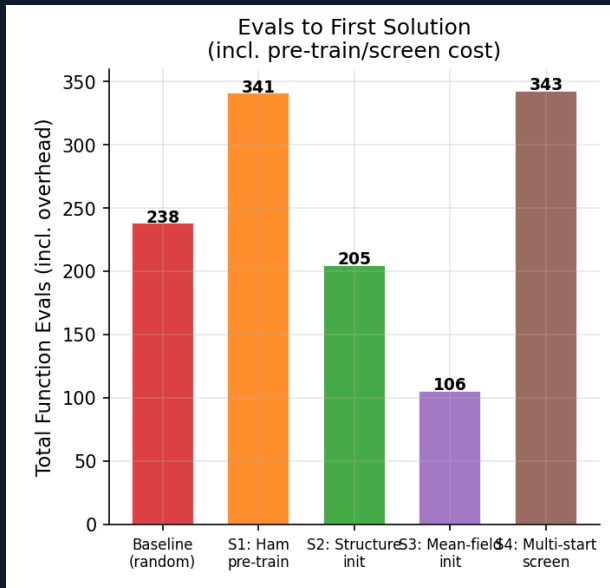
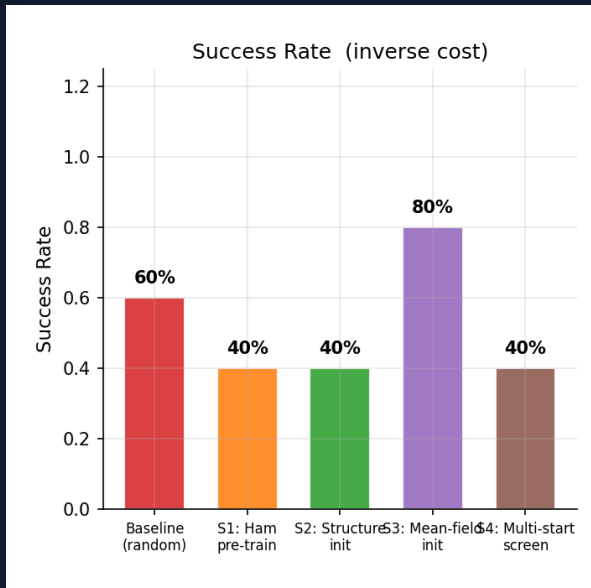
4

Generalization Beyond QUBO

The warm-start technique showing improvement in QUBO problems is referenced but its extension to inverse-type costs is explicitly left open.

Warm start strategies

→ $n = 4087 = 61 \times 67$ · $N = 16$ qubits · $\alpha = 0.25$ · 5 independent runs



S3 : Solves a classical approximation of the problem first → Estimate the probability of each bit being 0 or 1 → Ignore correlations between qubits → Choose probabilities that minimize the expected cost → Convert probabilities to rotation angles $\theta = \sin^{-1}(\sqrt{p^*})$ → Use these angles as initial parameters for VQE

Reference: Y. Chai, K. Jansen, S. Kühn, T. Schwägerl, and T. Stollenwerk, Structure-inspired ansatz and warm start of variational quantum algorithms for quadratic unconstrained binary optimization problems, arXiv:2407.02569.

BL Baseline (Random Init)

Reference

Random angles drawn from $U(-\pi, \pi)$. No prior knowledge. Used as the reference.

S1 Hamiltonian Pre-train

Cost Switching

Short COBYLA run on the squared-error Hamiltonian cost (easier landscape) → transfer final params to inverse-cost optimisation.

S2 Structure-Inspired Init

Analytical Bias

Layer-1 RY angles analytically biased toward $p \approx \sqrt{n}$ in binary. q-register set to uniform superposition ($\pi/2$). Deeper layers perturbed ± 0.25 .

S3 Mean-Field Init

★ Best Strategy

Classical mean-field optimisation over product-state marginals $P(\text{qubit}=1)$. RY angles set to $2 \cdot \arcsin(\sqrt{p^*})$. Best of 8 restarts selected.

S4 Multi-Start Screening

Screening

K=8 random starts, each run for 20 Hamiltonian-cost iterations. Best-scoring candidate params passed to full inverse-cost optimisation.

Thank You

VQE Prime Factorization — Replication & Extension Study

Sobhani, Chai, Hartung, Jansen — Phys. Rev. A 111, 042413 (2025)

DOI: [10.1103/PhysRevA.111.042413](https://doi.org/10.1103/PhysRevA.111.042413)